

FULL STACK APPLICATION DEVELOPMENT

SKILL – 3: Working with HQL - Sorting, Pagination & Aggregates

Name	S. LALITH ADITYA
ID	2400031810
Section	S-22
Subject	FULL STACK APPLICATION DEVELOPMENT

Aim

To perform advanced data retrieval operations using Hibernate Query Language (HQL) such as sorting, pagination, filtering, grouping, and aggregate functions.

Prerequisites

- Basic knowledge of Hibernate
- Basic idea of HQL syntax

Software Requirements

- Java 8
- MySQL
- Eclipse IDE
- Hibernate 5.6
- Maven

Task 1: Insert 5–8 Product Records

HQL / Hibernate Code:

```
session.save(product);
```

Output Screenshot:

id	description	name	price	quantity
3	Gaming Laptop	Laptop	2000000	12
4	Wireless Mouse	Mouse	1200	40
5	Gaming Laptop	Laptop	75000	10
6	Wireless Mouse	Mouse	1200	40
7	gaming	laptop	68000	7
8	normal	dell	78000	1
9	gaming	asus	98000	1
10	gaming	hp victus	10000	1
11	electronics	laptop	55000	5

Task 2: Sort Products by Price (Ascending)

from Product order by price asc

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit

2
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_ from product0_
Mouse 1200.0
Mouse 1200.0
hp victus 10000.0
laptop 55000.0
laptop 68000.0
Laptop 75000.0
dell 78000.0
asus 98000.0
Laptop 2000000.0

```

Task 3: Sort Products by Price (Descending)

from Product order by price desc

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit

3
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_ from product0_
Laptop 2000000.0
asus 98000.0
dell 78000.0
Laptop 75000.0
laptop 68000.0
laptop 55000.0
hp victus 10000.0
Mouse 1200.0
Mouse 1200.0

```

Task 4: Sort by Quantity (Highest First)

from Product order by quantity desc

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit

4
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_ from product0_
Mouse 40
Mouse 40
Laptop 12
Laptop 10
laptop 7
laptop 5
dell 1
asus 1
hp victus 1

```

Task 5: Pagination

```

Query q = session.createQuery("from Product");
q.setFirstResult(0);
q.setMaxResults(3);

```

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit

5
First 3 Products
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_ from product0_
com.klu.model.Product@31973858
com.klu.model.Product@67bb4dcd
com.klu.model.Product@738d37fc
Next 3 Products
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_0_ from product0_
com.klu.model.Product@1775c4e7
com.klu.model.Product@47829d6d
com.klu.model.Product@2f677247

```

Task 6: Aggregate Functions

```

select count(*) from Product
select min(price) from Product
select max(price) from Product

```

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit
6
Hibernate: select count(*) as col_0_0_ from product product0_
Hibernate: select count(*) as col_0_0_ from product product0_ where product0_.quantity>0
Hibernate: select product0_.description as col_0_0_, count(*) as col_1_0_ from product product0_ group by product0_.description
Hibernate: select min(product0_.price) as col_0_0_ from product product0_
Hibernate: select max(product0_.price) as col_0_0_ from product product0_
Total Products: 9
Quantity > 0: 9
Gaming Laptop 2
Wireless Mouse 2
gaming 3
normal 1
electronics 1
Min Price: 1200.0
Max Price: 200000.0

```

Task 7: Group By Description

select description, count(*) from Product group by description

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit
7
Hibernate: select product0_.description as col_0_0_, count(*) as col_1_0_ from product product0_ group by product0_.description
Gaming Laptop 2
Wireless Mouse 2
gaming 3
normal 1
electronics 1

```

Task 8: Filter Products Using WHERE

from Product where price between :min and :max

Output Screenshot:

```

1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit
8
Min price: 20000
Max price: 100000
Hibernate: select product0_.id as id1_0_, product0_.description as descript2_0_, product0_.name as name3_0_, product0_.price as price4_
com.klu.model.Product@7fda2001
com.klu.model.Product@11c713b7
com.klu.model.Product@2251b3bc
com.klu.model.Product@3f5e503c
com.klu.model.Product@3c232051

```

Task 9: LIKE Operations

name like 'A%'

name like '%p'

name like '%ou%'

length(name)=5

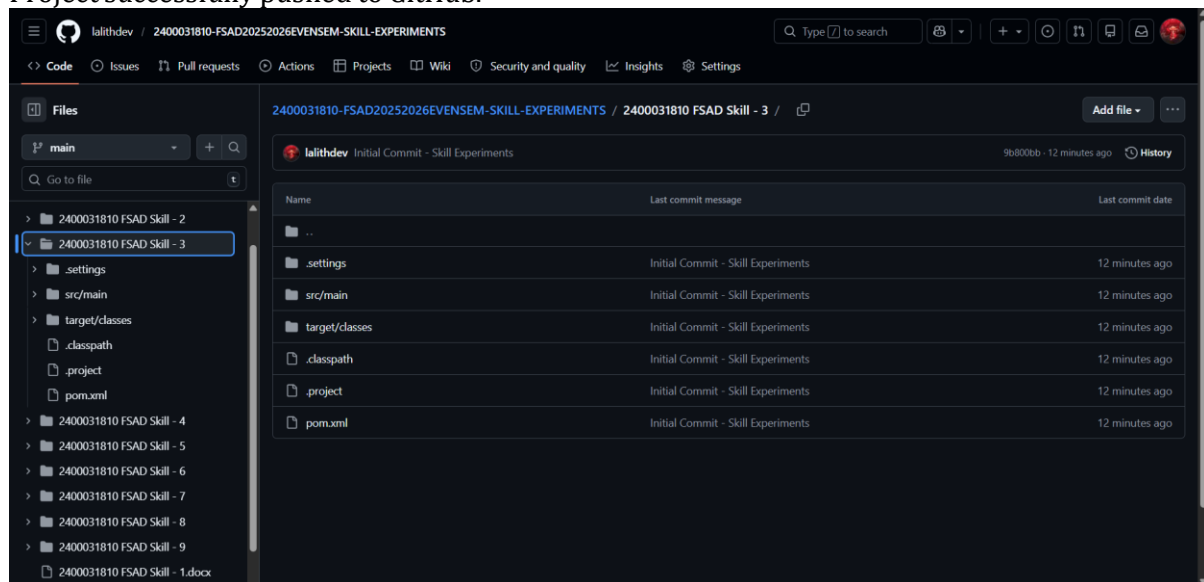
Output Screenshot:

```
1 Insert Product
2 Sort Price Ascending
3 Sort Price Descending
4 Sort Quantity Highest First
5 Pagination
6 Aggregate Functions
7 Group By Description
8 Filter Price Range
9 LIKE Operations
10 Exit

9
1 Starts With
2 Ends With
3 Contains
4 Exact Length
```

Task 10: GitHub Repository

Project successfully pushed to GitHub.



Result

All HQL operations including sorting, pagination, filtering, grouping and aggregate functions were successfully implemented using Hibernate.

Codes:

MainApp:

```
package com.klu.mainapp;
```

```
import com.klu.model.Product;
```

```
import org.hibernate.*;

import org.hibernate.cfg.Configuration;


import java.util.List;

import java.util.Scanner;


public class MainApp {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        SessionFactory sf = new Configuration().configure().buildSessionFactory();

        Session session = sf.openSession();

        Transaction tx = session.beginTransaction();


        System.out.println("1 Insert Product");

        System.out.println("2 Sort Price Ascending");

        System.out.println("3 Sort Price Descending");

        System.out.println("4 Sort Quantity Highest First");

        System.out.println("5 Pagination");

        System.out.println("6 Aggregate Functions");

        System.out.println("7 Group By Description");

        System.out.println("8 Filter Price Range");

        System.out.println("9 LIKE Operations");

        System.out.println("10 Exit");


        int ch = sc.nextInt();
```

```
switch (ch) {

    case 1:

        System.out.print("How many products: ");
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {

            Product p = new Product();

            System.out.print("Name: ");
            p.setName(sc.next());

            System.out.print("Description: ");
            p.setDescription(sc.next());

            System.out.print("Price: ");
            p.setPrice(sc.nextDouble());

            System.out.print("Quantity: ");
            p.setQuantity(sc.nextInt());

            session.save(p);
        }

        tx.commit();

        System.out.println("Products inserted");
        break;
```

case 2:

```
List<Product> asc =  
    session.createQuery("from Product order by price asc", Product.class).list();  
asc.forEach(p ->  
    System.out.println(p.getName() + " " + p.getPrice()));  
break;
```

case 3:

```
List<Product> desc =  
    session.createQuery("from Product order by price desc", Product.class).list();  
desc.forEach(p ->  
    System.out.println(p.getName() + " " + p.getPrice()));  
break;
```

case 4:

```
List<Product> qty =  
    session.createQuery("from Product order by quantity desc",  
Product.class).list();  
qty.forEach(p ->  
    System.out.println(p.getName() + " " + p.getQuantity()));  
break;
```

case 5:

```
Query q = session.createQuery("from Product");  
q.setFirstResult(0);  
q.setMaxResults(3);  
System.out.println("First 3 Products");  
q.list().forEach(System.out::println);
```

```

q.setFirstResult(3);
q.setMaxResults(3);
System.out.println("Next 3 Products");
q.list().forEach(System.out::println);
break;

```

case 6:

Long total =

```
(Long) session.createQuery("select count(*) from Product").uniqueResult();
```

Long available =

```
(Long) session.createQuery("select count(*) from Product where
quantity>0").uniqueResult();
```

List<Object[]> group =

```
session.createQuery("select description,count(*) from Product group by
description").list();
```

Double min =

```
(Double) session.createQuery("select min(price) from
Product").uniqueResult();
```

Double max =

```
(Double) session.createQuery("select max(price) from
Product").uniqueResult();
```

```
System.out.println("Total Products: " + total);
```

```
System.out.println("Quantity > 0: " + available);
```

```
group.forEach(o ->
```

```
System.out.println(o[0] + " " + o[1]));
```

```
System.out.println("Min Price: " + min);  
System.out.println("Max Price: " + max);  
  
break;
```

case 7:

```
List<Object[]> grp =  
    session.createQuery("select description,count(*) from Product group by  
description").list();  
grp.forEach(o ->  
    System.out.println(o[0] + " " + o[1]));  
  
break;
```

case 8:

```
System.out.print("Min price: ");  
  
double minp = sc.nextDouble();  
System.out.print("Max price: ");  
  
double maxp = sc.nextDouble();
```

Query range =

```
    session.createQuery("from Product where price between :a and :b");  
range.setParameter("a", minp);  
range.setParameter("b", maxp);  
  
range.list().forEach(System.out::println);  
  
break;
```

case 9:

```
System.out.println("1 Starts With");  
  
System.out.println("2 Ends With");
```

```
System.out.println("3 Contains");
```

```
System.out.println("4 Exact Length");
```

```
int like = sc.nextInt();
```

```
if (like == 1) {
```

```
    System.out.print("Enter letters: ");
```

```
    String s = sc.next() + "%";
```

```
    session.createQuery("from Product where name like :x")
```

```
        .setParameter("x", s)
```

```
        .list().forEach(System.out::println);
```

```
}
```

```
if (like == 2) {
```

```
    System.out.print("Enter letters: ");
```

```
    String s = "%" + sc.next();
```

```
    session.createQuery("from Product where name like :x")
```

```
        .setParameter("x", s)
```

```
        .list().forEach(System.out::println);
```

```
}
```

```
if (like == 3) {
```

```
    System.out.print("Enter letters: ");
```

```
    String s = "%" + sc.next() + "%";
```

```
    session.createQuery("from Product where name like :x")
```

```
        .setParameter("x", s)
```

```
        .list().forEach(System.out::println);
```

```
}
```

```

        if (like == 4) {
            System.out.print("Enter length: ");

            int len = sc.nextInt();

            session.createQuery("from Product where length(name)=:l")
                .setParameter("l", len)
                .list().forEach(System.out::println);
        }

        break;

    case 10:
        System.out.println("Program Ended");

        break;
    }

    session.close();
    sf.close();
}
}

```

Product.java:

```

package com.klu.model;

import javax.persistence.*;

@Entity
@Table(name = "product")

```

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private String description;
    private double price;
    private int quantity;

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public int getQuantity() {
```

```
    return quantity;
}
```

```
public void setName(String name) {
    this.name = name;
}
```

```
public void setDescription(String description) {
    this.description = description;
}
```

```
public void setPrice(double price) {
    this.price = price;
}
```

```
public void setQuantity(int quantity) {
    this.quantity = quantity;
}
}
```

Hibernate.cfg.xml:

```
package com.klu.model;
```

```
import javax.persistence.*;
```

```
@Entity
```

```
@Table(name = "product")
```

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private String description;
    private double price;
    private int quantity;

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public int getQuantity() {
```

```

        return quantity;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}

```

Pom.xml:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

```

<modelVersion>4.0.0</modelVersion>

<groupId>com.klu</groupId>

<artifactId>HibernateSkill3</artifactId>

<version>1.0</version>

<properties>

 <maven.compiler.source>1.8</maven.compiler.source>

 <maven.compiler.target>1.8</maven.compiler.target>

</properties>

<dependencies>

 <dependency>

 <groupId>org.hibernate</groupId>

 <artifactId>hibernate-core</artifactId>

 <version>5.6.15.Final</version>

 </dependency>

 <dependency>

 <groupId>javax.persistence</groupId>

 <artifactId>javax.persistence-api</artifactId>

 <version>2.2</version>

 </dependency>

 <dependency>

 <groupId>mysql</groupId>

 <artifactId>mysql-connector-java</artifactId>

<version>8.0.33</version>

</dependency>

<dependency>

<groupId>org.slf4j</groupId>

<artifactId>slf4j-simple</artifactId>

<version>1.7.36</version>

</dependency>

</dependencies>

</project>